

Modern Operating Systems
Chapter 5 – Input/Output
Software

Zhang Yang
Spring 2025



Contents of the Lecture

- 5.2 Principles of I/O Software
- 5.3 I/O Software Layers



Principles of I/O Software

■ Goals of I/O Software

- ☐ Device Independence
- ☐ Uniform Naming
- ☐ Error Handling
- ☐ Synchronous vs. Asynchronous Transfer
- ☐ Buffering
- ☐ Sharable vs. Dedicated Devices

■ I/O Operation

- ☐ Programmed I/O
- ☐ Interrupt-driven I/O
- ☐ I/O Using DMA

Goals of I/O Software (1)

■ Device Independence

- It means that programs can access any I/O device without specifying device in advance. (Read and Write from Floppy, Disk, CD-ROM without modifying.)
- I/O software should abstract the device details.
- E.g. sort <input> output
 - Have it work with input coming from a floppy disk, an IDE disk, a SCSI disk, or the keyboard, and the output going to any kind of disk or the screen.
- It is up to the operating system to take care of the problems caused by the fact that these devices really are different and require very different command sequences to read or write.

Goals of I/O Software (2)

■ Uniform Naming

- The way we specify a file or device should be done in the same way.
- Name of a file or device is a string or an integer, and not depend on the device.
 - Different devices of same type have similar name.
- In UNIX, all I/O is integrated with the file system.



Goals of I/O Software (3)

■ Error Handling

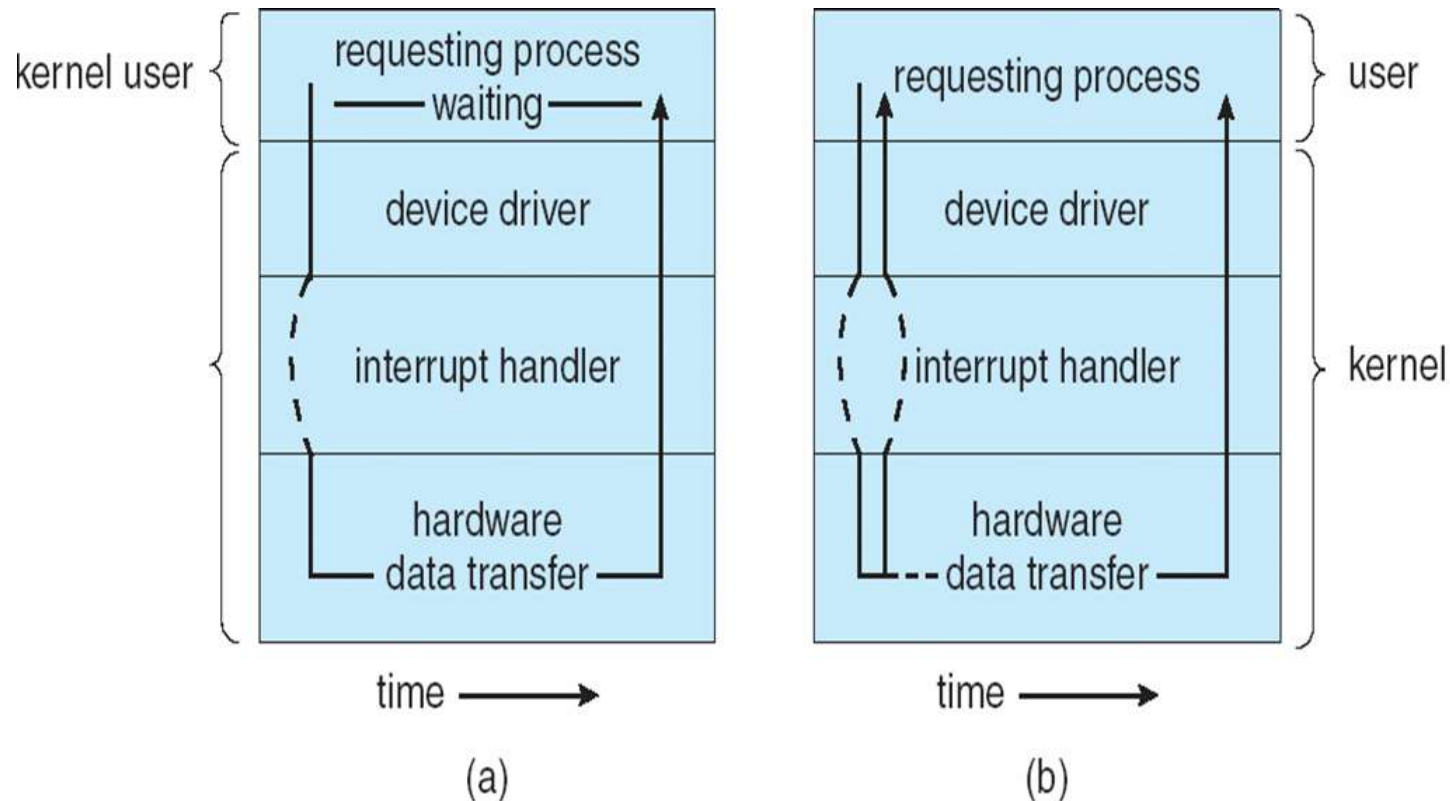
- ☐ Handle the errors close to the hardware as possible.
- ☐ Propagate errors up only when lower layer cannot handle it.
- ☐ Hide errors as much as possible—many HW errors are transient.

Goals of I/O Software (4)

- Synchronous vs. Asynchronous Transfers
 - Blocking transfers vs. Interrupt-driven
 - Most I/O hardware operates asynchronously.
 - Synchronous read/write at application level.
 - Synchronous (blocking) read/write easier to program.
 - Some very high-performance applications need to control all the details of the I/O, so some operating systems make asynchronous I/O available to them.
 - It is up to OS to make I/O operations that are actually interrupt-driven look blocking to the user program.

Goals of I/O Software (5)

■ Synchronous vs. Asynchronous Transfers (ctd.)



Blocking/Synchronous I/O Interrupt-driven/Asynchronous I/O

Goals of I/O Software (6)

■ Buffering

- Buffering is done to cope with a speed mismatch between the producer and consumer of a data stream or to adapt between devices that have different data transfer sizes.

■ Sharable vs. Dedicated Devices

- E.g. Disks are sharable.
- E.g. Tape drives are dedicated, may introduce a variety of problems, such as deadlock.
- OS must be able to handle both shared and dedicated devices in a way that avoids problem.
 - Make dedicated devices appear sharable. E.g printers



I/O Software

- System call in user-space.
- Data is copied from user space to kernel space.
- I/O software can operate in several modes.
 - Programmed I/O
 - Polling/Busy waiting for the device.
 - Interrupt-Driven I/O
 - Operation is completed by interrupt routine.
 - DMA-based I/O
 - Set up controller and let it deal with the transfer.



I/O Software Layers

- Overview of the I/O Software
- Interrupt Handlers
- Device Drivers
- Device Independent I/O Software
- User Space I/O Software

Programmed I/O (1)

- CPU is always busy with I/O until I/O gets completed.
 - Fine for single process systems.
 - MS-DOS
 - Embedded systems
 - But not a good approach for multi-programming and time-sharing systems.

Programmed I/O (2)

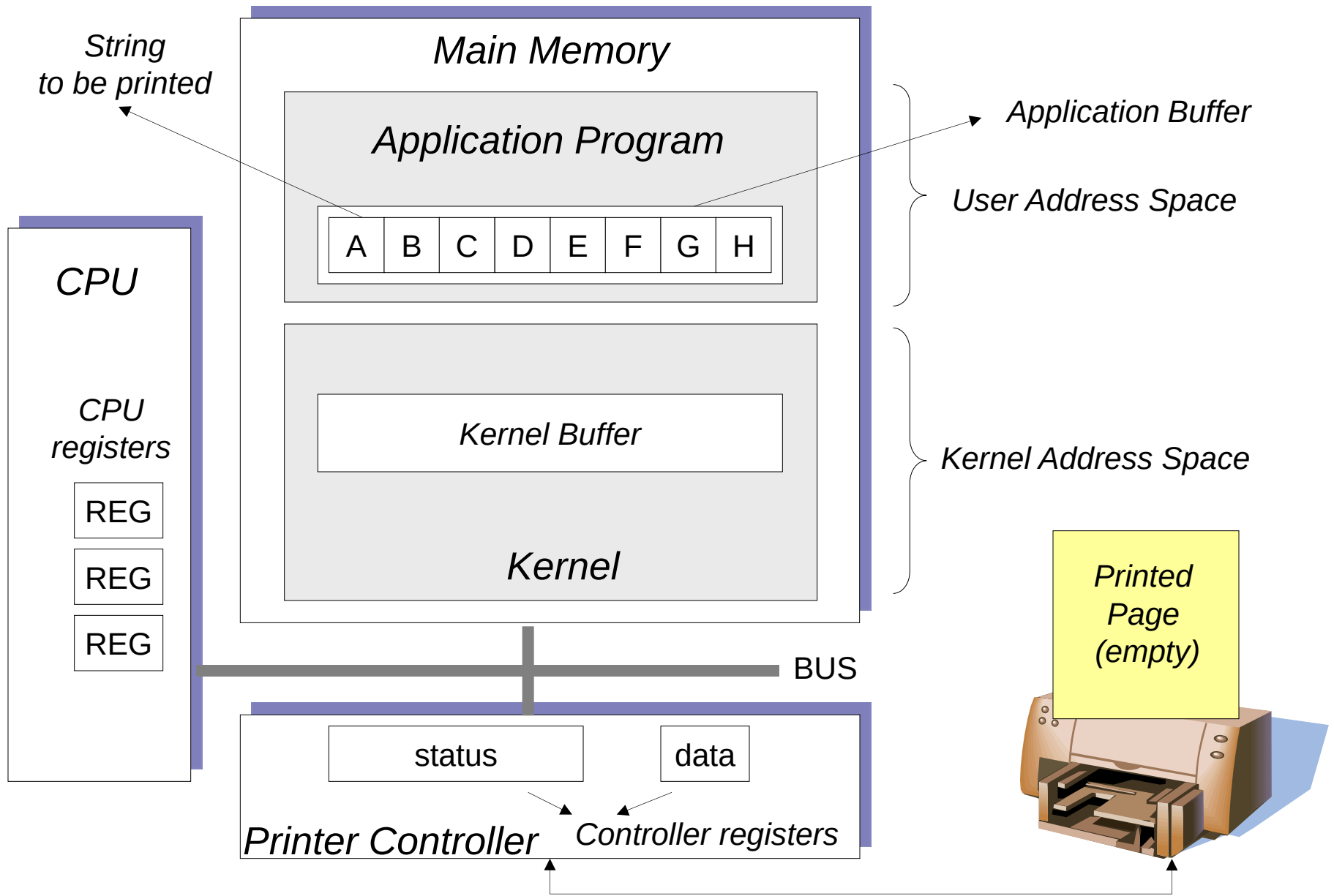
- Programmed I/O Operations (output)
 - Data copied to kernel space from user space.
 - OS then enters a tight loop (Polling) the device (usually checking a status register) to see if the device is ready for more data.
 - Once the device is ready to accept more data, the OS will copy the data to the device register.
 - Once all data has been copied control is returned to the user program.

Programmed I/O (3)

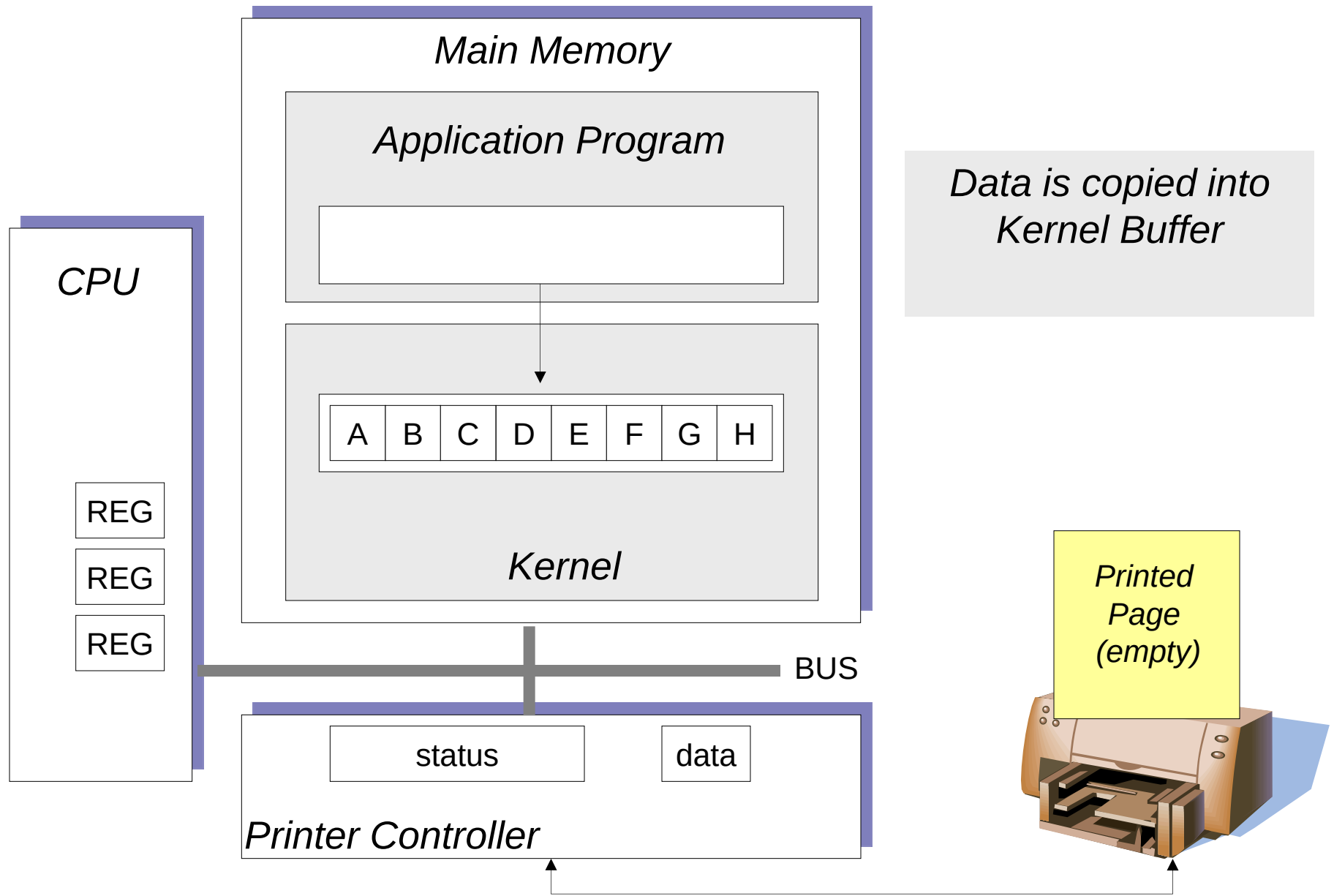
■ Printing String Example

- Assume a program that wants to print a string to a printer.
- Assume the string is “ABCDEFGH”.
 - 8 character long string.
- Assume printer has 1 byte of data buffer to put the character that needs to be printed.

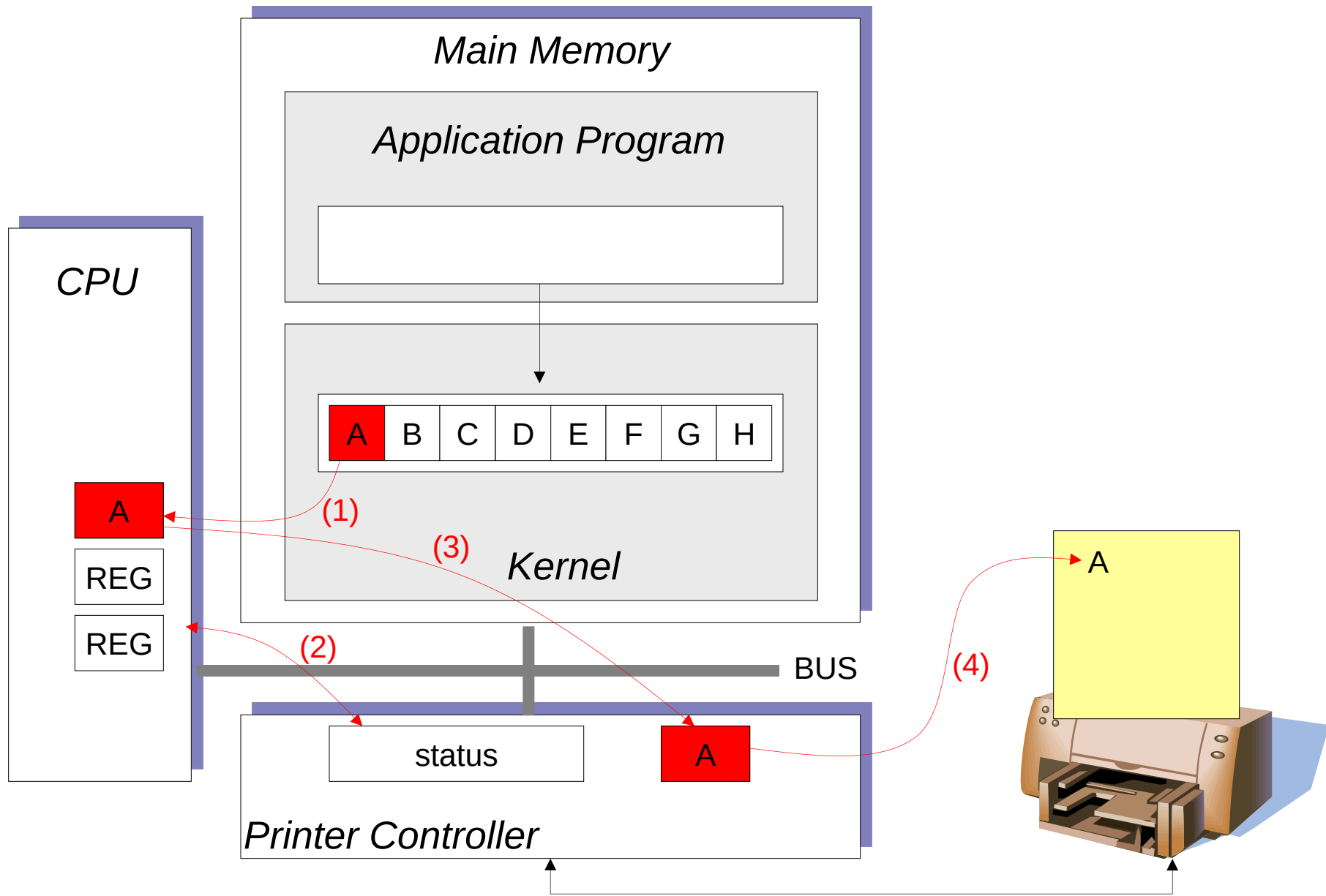
Programmed I/O (4)



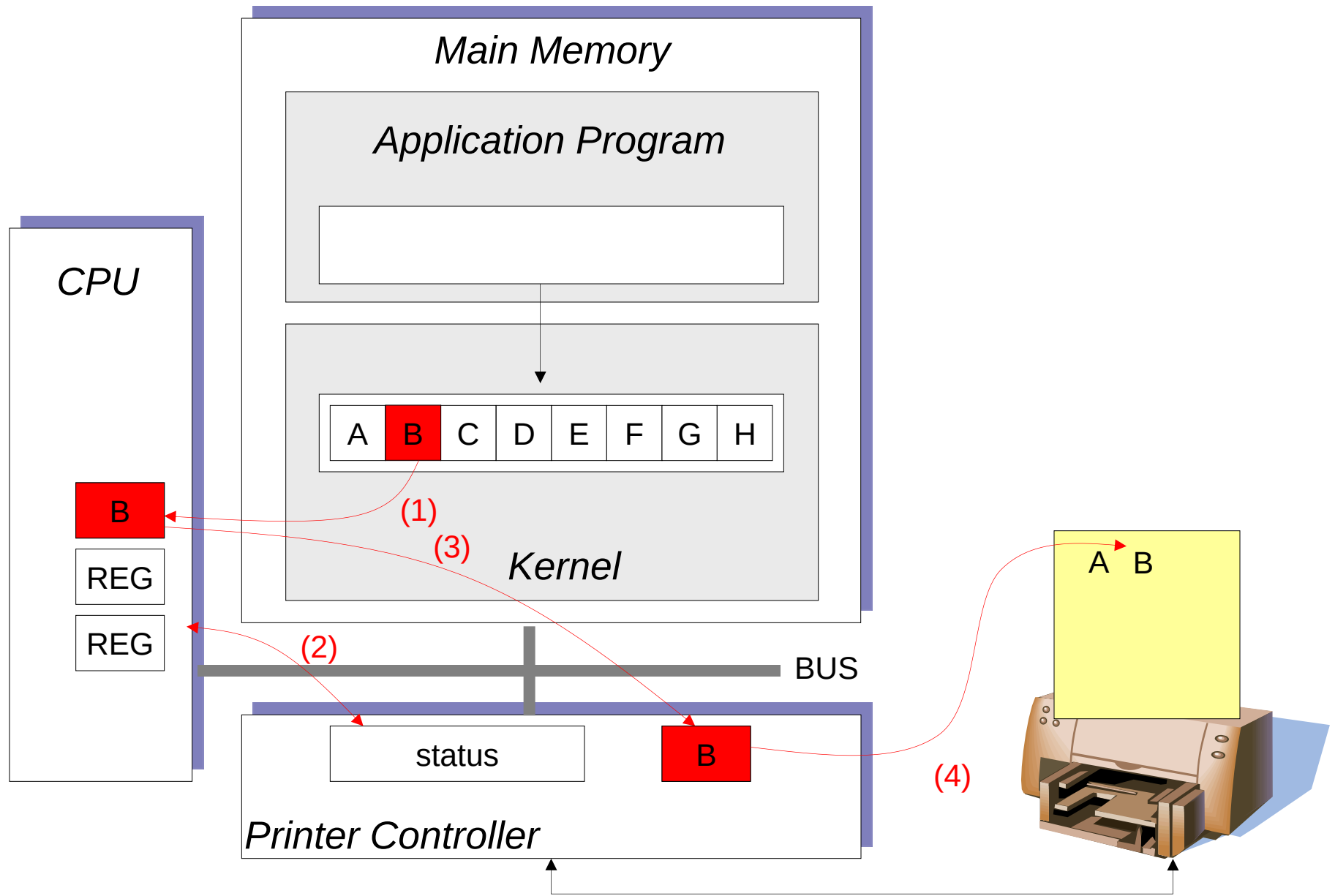
Programmed I/O (5)



Programmed I/O (6)



Programmed I/O (7)

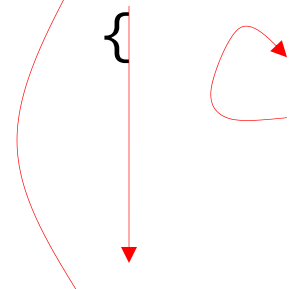


Programmed I/O (8)

■ Printing String Example (ctd.)

□ How would we implement copy in kernel?

```
/*
    buffer is user buffer
    p is kernel buffer
    count is number of bytes to be copied
*/
copy_from_user(buffer, p, count);
for (i=0; i < count; ++i) /* loop in every character */
{
    while (*printer_status_register != READY)
    {} /* busy waiting - loop until ready */
    *printer_data_register = p[i]; /* output one
                                     character */
}
return_to_user();
```

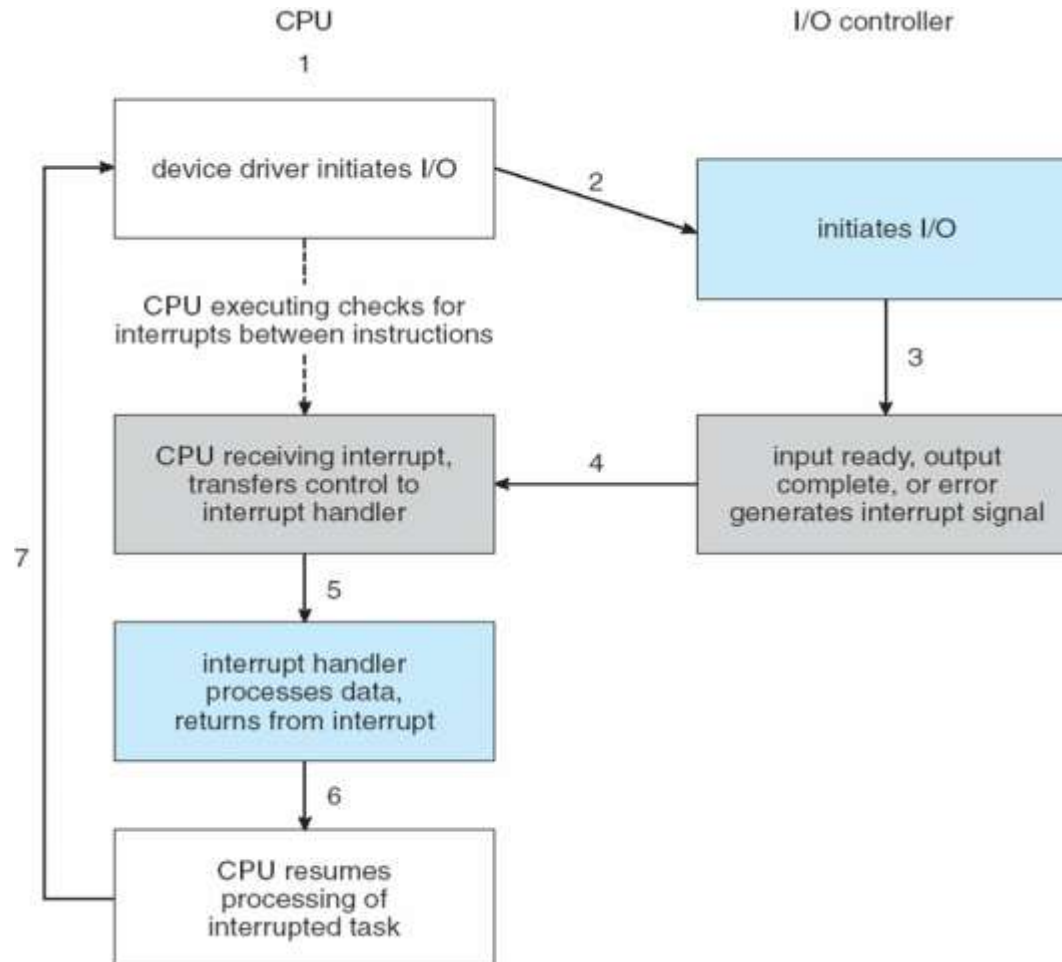


Programmed I/O (9)

- What is good about programmed I/O?
 - Usually simple to implement.
- What is bad about programmed I/O?
 - Since we are tying up the CPU checking to see if we can access a device, we are in a state of "busy waiting". No "real" work is being done.
 - Assume printer prints at a rate 100 chars/second.
 - Each character takes 10ms to print.
 - During this 10 ms, CPU will be busy with checking the status register of printer (controller).
 - This is waste of CPU.
 - During 10ms period something else can be made (Another process can be run).

Interrupt Driven I/O (1)

■ Interrupt Driven I/O Cycle



Interrupt Driven I/O (2)

■ Printing String Example

- The user program makes a system call telling the OS to print the string.
- After copying application buffer content into kernel buffer, and sending 1 character to printer:
 - CPU does not wait until printer is *READY* again.
 - Instead the scheduler() is called and some other process is run.
 - The current process is put into *blocked* state until the entire string is printed.
 - Printer controller generates an hardware interrupt when the printer has printed the character and is ready again.

Interrupt Driven I/O (3)

■ Printing String Example (ctd.)

- Code executed in kernel when the print() system call is made by the application process.

```
copy_from_user(buffer, p, count);
enable_interrupts();
while (*printer_status_reg != READY)
{
}; /* loop until printer controller becomes READY */
*printer_data_register = p[0]; // send to the first character
                                to the printer controller */
scheduler(); /* do some other task */
```

Interrupt Driven I/O (4)

■ Printing String Example (ctd.)

- Interrupt Service Routine that is executed when the printer controller becomes READY for one more byte again.

```
If (count == 0) {    /* we are finishes printing the string */
    unblock_user(); /* unblock the user process that
                       wanted to print the string */
} else {
    /* copy one more character to the controller buffer */
    *print_data_register = p[i];
    count = count - 1;
    i = i + 1;
}
acknowledge interrupt();
return_from_interrupt(); /* finished serving the interrupt */
```

Interrupt Driven I/O (5)

- Advantages of Interrupt-Driven I/O
 - Its faster than Programmed I/O.
 - Efficient too.
- Disadvantages of Interrupt-Driven I/O
 - It can be tricky to write if using a low level language.
 - Interrupt occurs on every character. It requires the CPU to get involved for every word (character) of data that goes from memory to I/O module or from I/O module to memory, and interrupt processing then becomes expensive.

I/O Using DMA (1)

- In essence, DMA is programmed I/O, only with the DMA controller doing all the work, instead of the main CPU.
- Printing String Example
 - DMA controller will feed the characters from kernel buffer to the printer controller.
 - CPU is not bothered during this transfer.
 - After transfer of whole buffer (string) is completed, then the CPU is interrupted.

I/O Using DMA (2)

■ Printing String Example (ctd.)

- *Code executed in Kernel when the print() system call is made by the application process.*

```
copy_from_user(buffer, p, count);  
setup_DMA_controller();  
scheduler();
```

- Interrupt Service Routine that is executed when a 'transfer completed interrupt' is received from DMA.

```
acknowledge_interrupt();  
unblock_user();  
return_from_interrupt();
```



I/O Using DMA (3)

- What is good about I/O using DMA?
 - Reducing the number of interrupts from one per character to one per buffer printed.
- What is bad about I/O using DMA?
 - The DMA controller is usually much slower than the main CPU.



Overview of the I/O Software (1)

- The I/O component of the OS is organized in layers.
 - Interrupt Handlers
 - Device Drivers
 - Device-independent I/O
 - User-level I/O System Calls
- Abstraction, encapsulation and layering
 - Any complex software engineering problem.
 - Layers can be modified independently without affecting layers above and below.

Overview of the I/O Software (2)

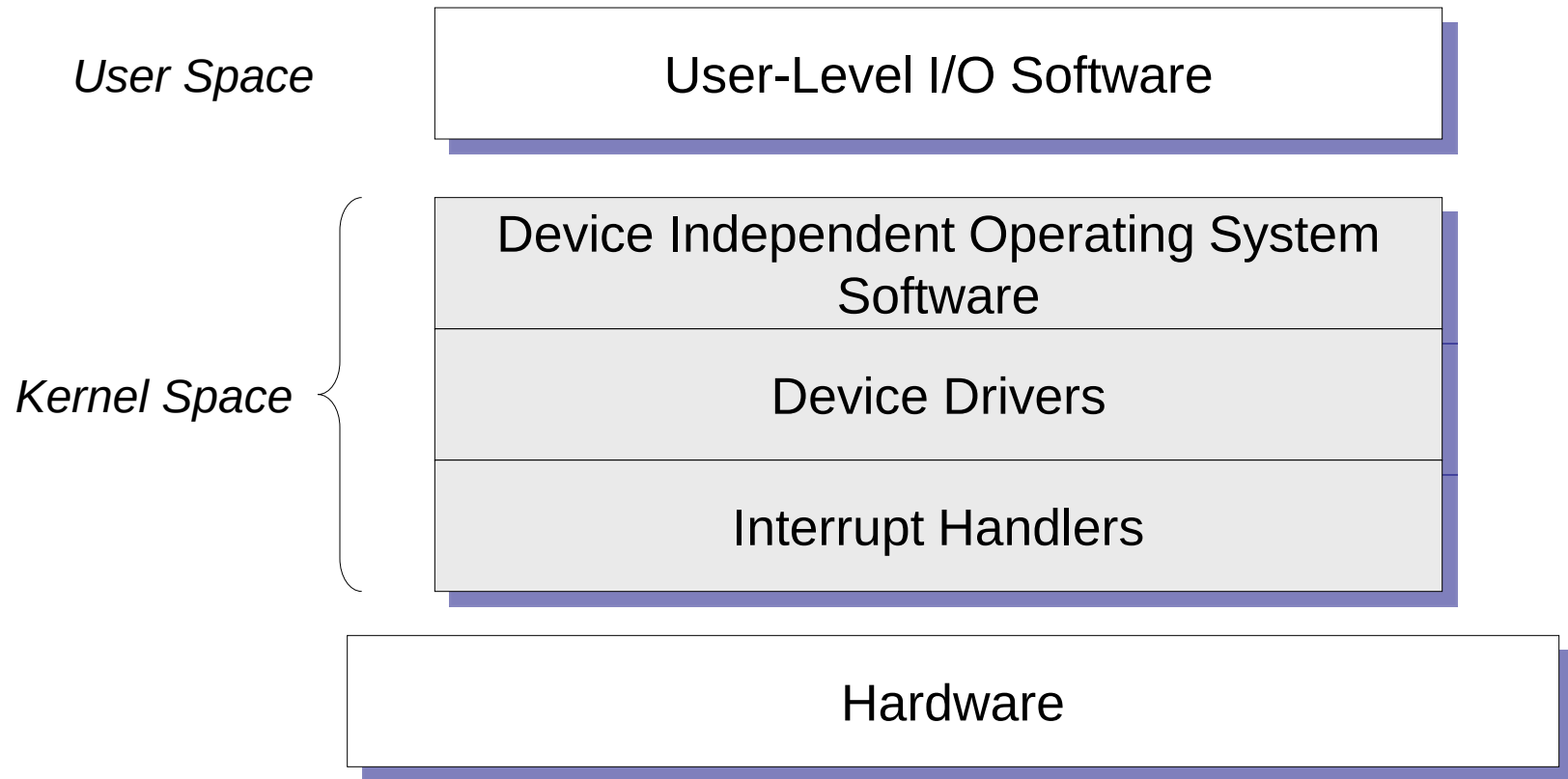


Figure 5-11 Layers of the I/O Software System

Interrupt Handlers (1)

- Can execute at (almost) at any time.
- Interrupt handlers are best hidden.
 - So as little of OS know about it as possible.
- Best way to hide is to have the driver starting an I/O operation block until interrupt notifies of completion.
 - A *down* on a semaphore
 - A *wait* on a condition variable
 - A *receive* on a message
 - ...
- Interrupt handler does the actual work and then unblocks driver that started it.
- Mechanism works best if device drivers are threads in the kernel.

Interrupt Handlers (2)

■ Interrupt Handler Routines

- Interrupts (asynchronous, external to process) basically use the same mechanism as exceptions and traps (synchronous, internal to process).
- When an interrupts happen, the CPU saves a small amount of state and jumps to an interrupt-handler routine at a fixed address in memory.
- The interrupt routine's location is determined by an interrupt vector.

Interrupt Handlers (3)

■ Interrupt Handler Routines (ctd.)

□ Intel Pentium processor event-vector table

Non-maskable,
used for
various error
conditions

Maskable,
used for
device-
generated
interrupts

vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19-31	(Intel reserved, do not use)
32-255	maskable interrupts

Interrupt Handlers (4)

■ Interrupt Handler Steps

- Save Registers not already saved by hardware interrupt mechanism.
- (Optionally) Set up context for interrupt service procedure. E.g. TLB, MMU, and page table
 - Typically, handler runs in the context of the currently running process.
 - No expensive context switch.
- Set up stack for interrupt service procedure
 - Handler usually runs on the kernel stack of current process.
- Ack/Mask interrupt controller, re-enable other interrupts
 - What does this imply?

Interrupt Handlers (5)

■ Interrupt Handler Steps (ctd.)

- Run interrupt service procedure
 - Acknowledges interrupt at device level.
 - Figures out what caused the interrupt.
 - Received a network packet, disk read finished, UART transmit queue Empty
 - If needed, it signals blocked device drive.
- In some cases, will have woken up a higher priority blocked thread.
 - Choose newly woken thread to schedule next.
 - Set up MMU context for process to run next.
 - What if we are nested?
- Load new/original process' registers
- Re-enable interrupt; Start running the new process.

Device Drivers (1)

- A device driver is a specific module that manages the interaction between the device controller and the OS.
 - device independent request -> device driver -> device dependent request
- Generally written by device manufacturers.
 - Not necessarily by OS writers.
- A driver can be written for more than one OS.
- Each driver can handle one type of device or one class (e.g. SCSI)



Device Drivers (2)

- Driver usually part of OS kernel
 - Unix – kernel compiled with device drivers.
 - Linux – uses loadable modules which are chunks of code loaded into kernel while system is running.
 - Windows – dynamically loaded into system at startup and plug-n-play.

Device Drivers (3)

- A driver has several functions:
 - Accept abstract read/write requests from the device-independent software above and translate them into concrete I/O-module-specific commands.
 - Schedule requests: Optimize queued request order for best device utilization (eg: disk arm).
 - Initialize the device, if needed.
 - Manage data transfer.
 - Maintain the integrity of driver and kernel data structures.
 - Manage power requirements and log device events.

Device Drivers (4)

- Typical code organization of a device driver
 - Check validity of input parameters coming from above.
 - If valid, translate to concrete commands, e.g., convert block number to head, track & sector in a disk's geometry.
 - Check if device currently in use; if yes, queue request; if not, possibly switch device on, warm up, initialize, etc.
 - Issue appropriate sequence of commands to controller.
 - If needs to wait, block.
 - Upon interrupted from blocking, check for errors and pass data back.
 - Process next queued request.

Device Drivers (5)

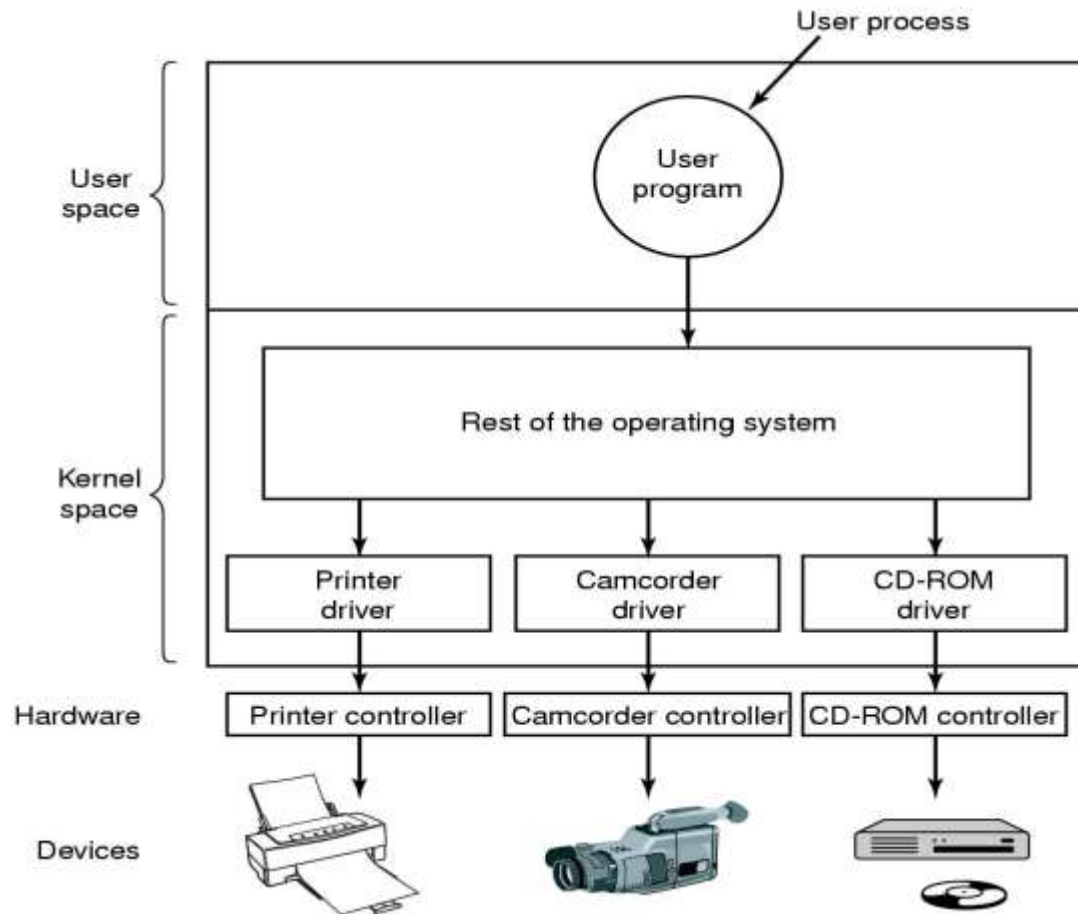


Figure 5-12 Logical position of device drivers is shown here
Communications between drivers and device controllers goes over the bus

Device Drivers (6)

■ Some Other Issues

- A driver code must be reentrant as they can be called by another process while a process is already blocked in the driver.
 - Re-entrant: Code that can be executed by more than one thread (or CPU) at the same time.
- A driver code needs to handle pluggable devices.
- Drivers are not allowed to make system calls, but they may call kernel procedures.
 - E.g. Manage MMU, timers, DMA controller, interrupt controller, etc.



Device-Independent I/O Software

- Boundary between device driver and device-independent software varies between systems and devices.
- Device-Independent I/O Software Functions
 - Uniform interfacing to device drivers
 - Buffering
 - Error reporting
 - Allocating and releasing dedicated devices
 - Providing a device-independent block size

Function - Uniform Interfacing (1)

- The interface between the device driver and the rest of the OS.

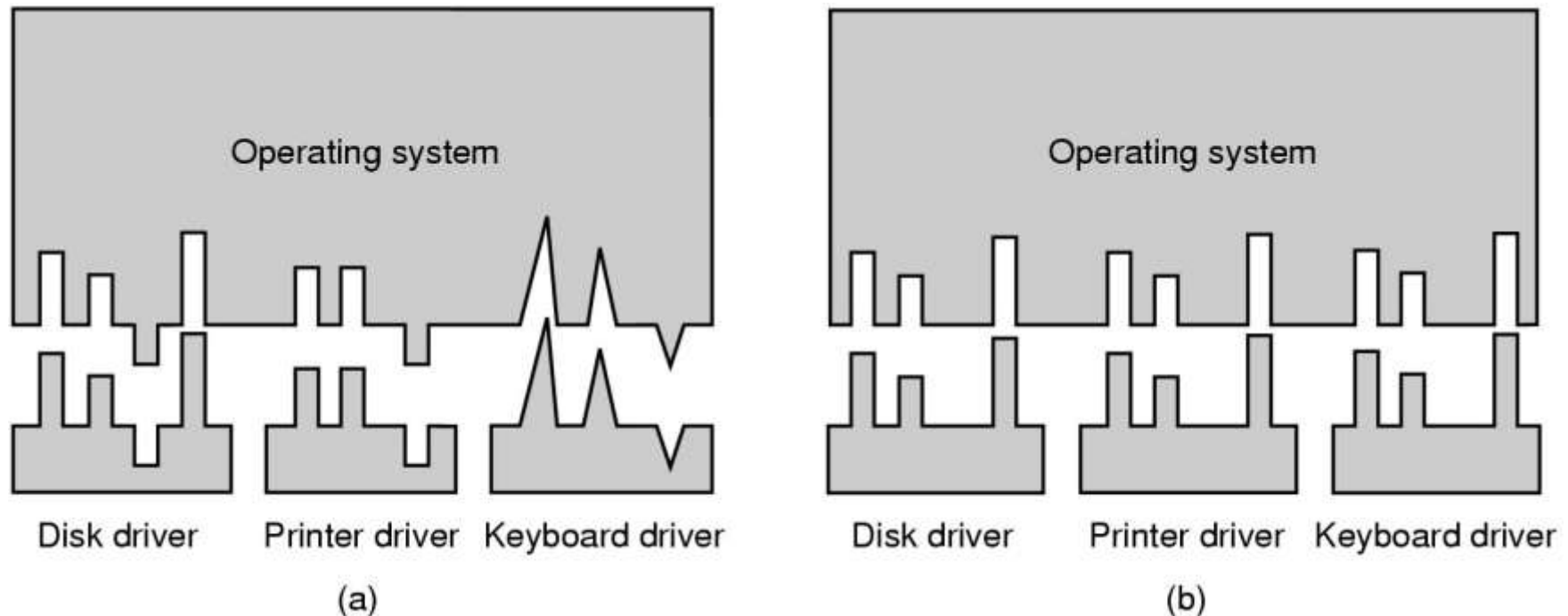


Figure 5-14 (a) Without a standard driver interface
(b) With a standard driver interface

Function - Uniform Interfacing (2)

- Uniform device interface for kernel code
 - Allows different devices to be used the same way.
 - No need to rewrite file-system to switch between SCSI, IDE or RAM disk.
 - Allows internal changes to device driver without fear of breaking kernel code.
- Uniform kernel interface for device code
 - Drivers use a defined interface to kernel services (e.g. kmalloc, install IRQ handler, etc.)
 - Allows kernel to evolve without breaking existing drivers.
- Together both uniform interfaces avoid a lot of programming implementing new interfaces.

Function - Uniform Interfacing (3)

- How I/O devices are named?
 - Device-independent software maps symbolic device names onto the proper driver.
 - In Unix, devices are modeled as special files.
 - They are accessed through the use of system calls such as `open()`, `read()`, `write()`, `close()`, `ioctl()`, etc.
 - A file name is associated with each device.
 - The usual protection rules for files also apply to I/O devices.
 - Example
 - Major device number locates the appropriate driver.
 - Minor device number (stored in i-node) is passed as a parameter to the driver in order to specify the unit to be read or written.

Function - Uniform Interfacing (4)

i-node

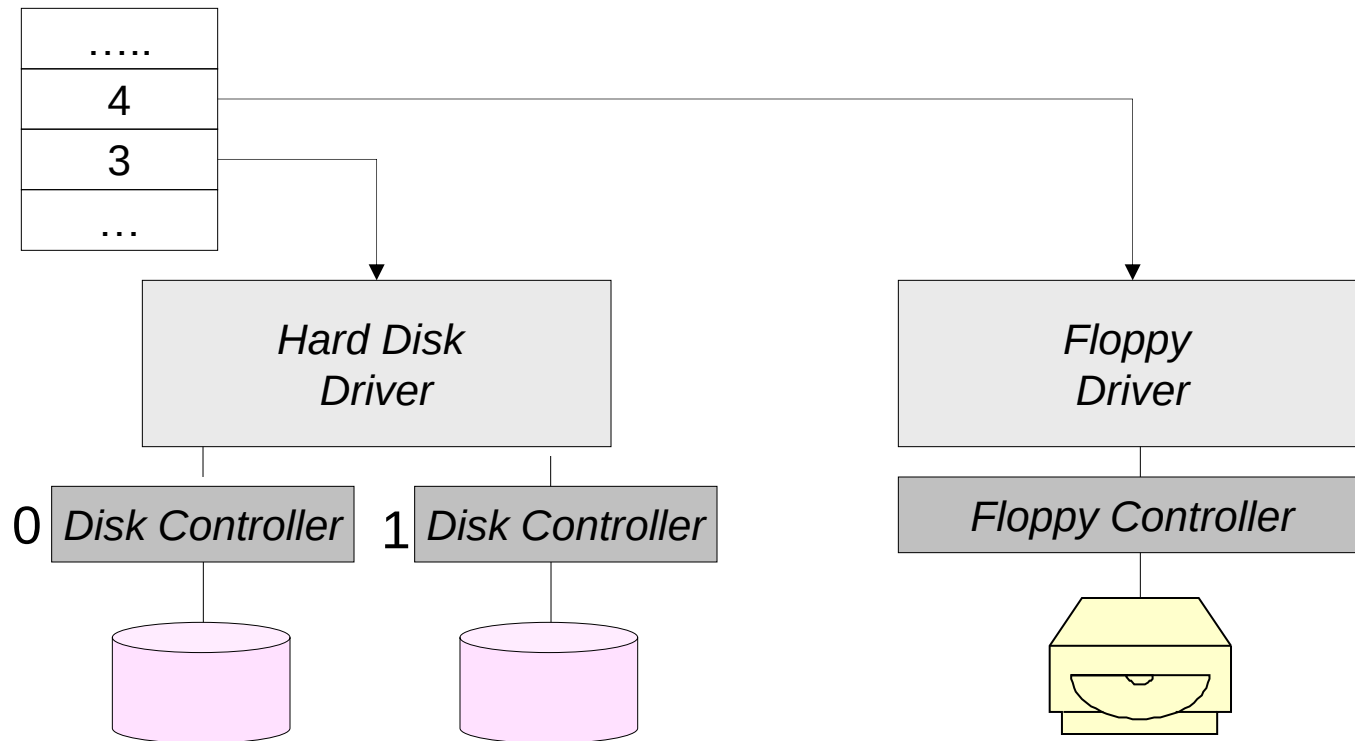
name: /dev/disk0
major number: 3
minor number: 0

i-node

name: /dev/disk1
major number: 3
minor number: 1

i-node

name: /dev/fd0
major number: 4
minor number: 0



Mapping symbolic I/O device names to their appropriate drivers

Function - Buffering (1)

- Memory area that stores data in kernel space while transferred between device and application.
- Cope with a speed mismatch between producer and consumer.
 - E.g. Modem thousand times slower than disk.
- Adapt between services with different data-transfer sizes.
 - E.g. Fragmentation and reassembly of network packets.

Function - Buffering (2)

- Example: consider a process that wants to read data from a modem.

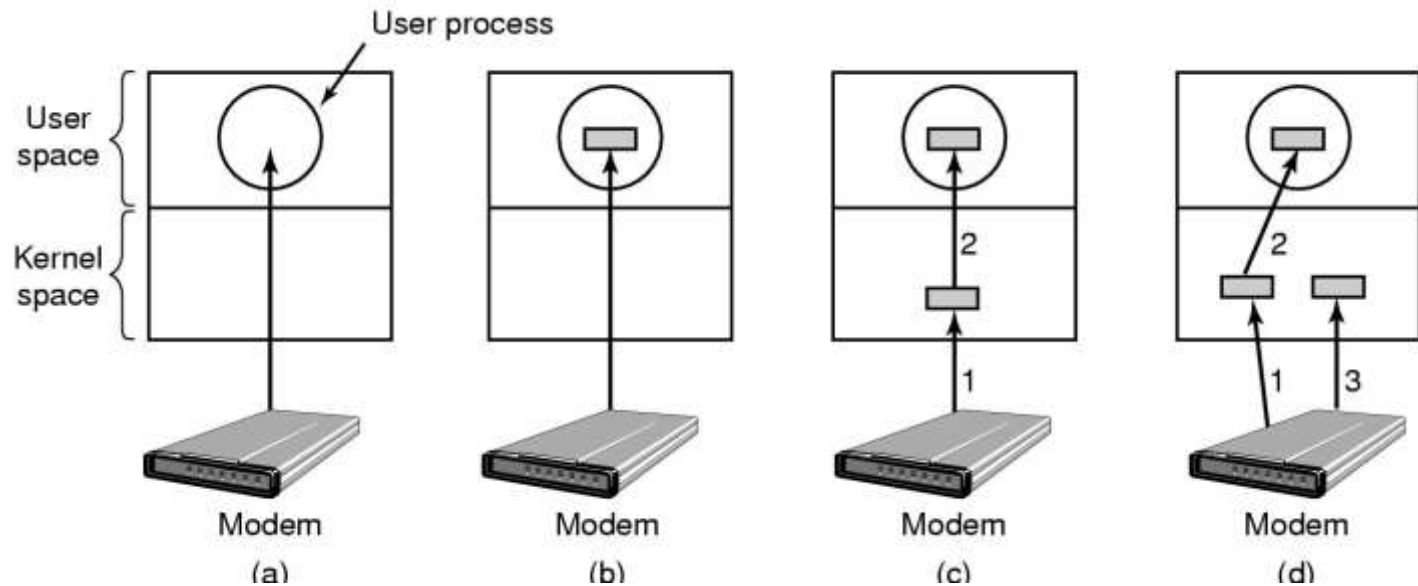


Figure 5-15

(a) Unbuffered input

(b) Buffering in user space

(c) Buffering in the kernel followed by copying to user space

(d) Double buffering in the kernel

Function - Buffering (3)

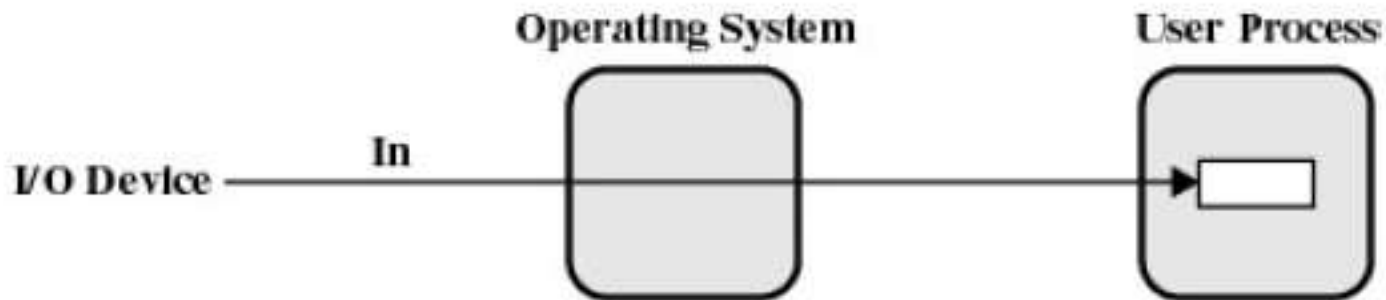
■ No Buffering

- Process must read/write a device a byte/word at a time.
 - Each individual system call adds significant overhead.
 - Process must wait until each I/O is complete.
 - Blocking/interrupt/waking adds to overhead.
 - Many short runs of a process is inefficient (poor CPU cache temporal locality).

Function - Buffering (4)

■ User-level Buffering

- Process specifies a memory buffer that incoming data is placed in until it fills.
 - Filling can be done by interrupt service routine.
 - Only a single system call, and block/wakeup per data buffer.
 - Much more efficient



Function - Buffering (5)

■ User-level Buffering (ctd.)

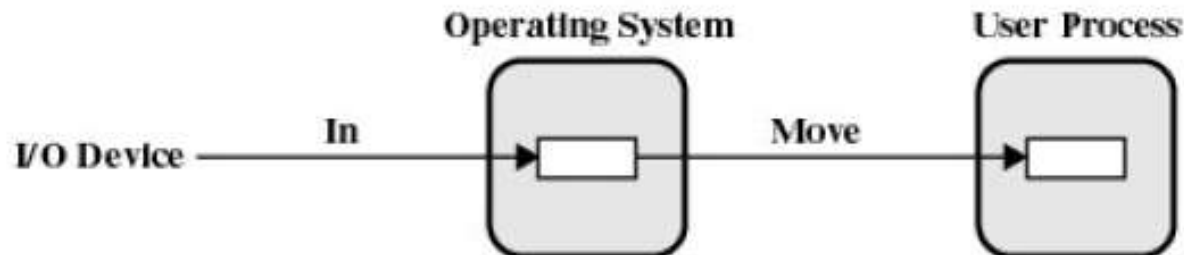
□ Issue

- What happens if buffer is paged out to disk?
 - Could lose data while buffer is paged in.
 - Could lock buffer in memory (needed for DMA), however many processes doing I/O reduce RAM available for paging. Can cause deadlock as RAM is limited resource.

Function - Buffering (6)

■ Single Buffer

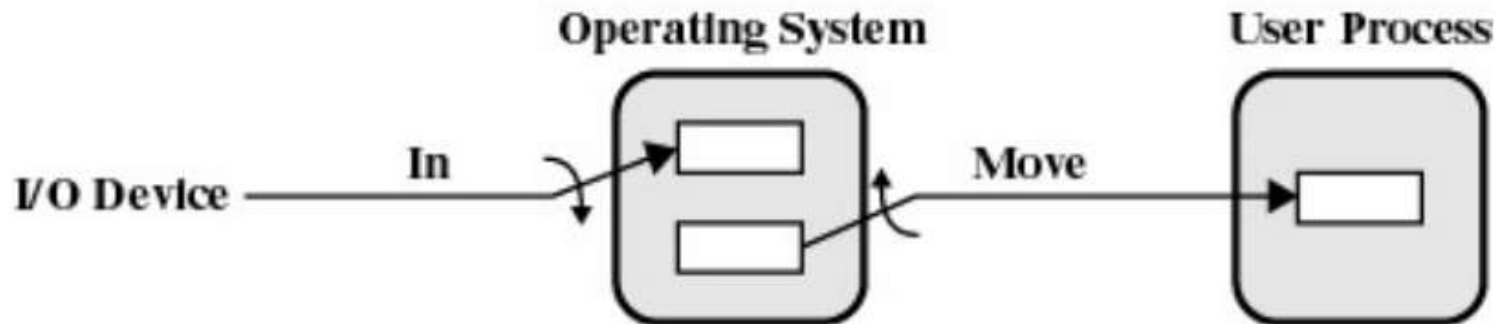
- Operating system assigns a buffer in main memory for an I/O request.
- What happens if kernel buffer is full, the user buffer is swapped out, and more data is received???
- We start to lose characters or drop network packets



Function - Buffering (7)

■ Double Buffer

- Use two system buffers instead of one.
- A process can transfer data to or from one buffer while the operating system empties or fills the other buffer.



Function - Buffering (8)

■ Circular Buffer

- More than two buffers are used.
- Each individual buffer is one unit in a circular buffer.
- Used when I/O operation must keep up with process.



- Notice that buffering, double buffering, and circular buffering are all Bounded Buffer Problem (Producer-Consumer Problem).

Function - Error Reporting

- Some errors are handled by device controllers.
 - E.g., Checksum incorrect, re-correct the block by using redundant bits.
- Some by device drivers.
 - E.g., disk block could not be read, re-issue the read operation for block from disk.
- Some by the device-independent software layer of OS.
 - Retry I/O or report to user with error code.
 - Identifies programming error vs. I/O error
 - Programming errors
 - E.g., Attempt to write to a read-only device.
 - Actual I/O errors
 - E.g., The camcorder is shut-off, therefore we could not read. Return an indication of this error to the calling application.

Function - Allocating& Releasing Dedicated Devices

- Do not allow concurrent accesses to dedicated devices such as CD-RWs.
- Require processes to perform `open()`'s on the special files for devices directly.
 - The process retries if `open()` fails.
- Have special mechanisms for requesting and releasing dedicated devices.
 - An attempt to acquire a device that is not available blocks the caller.

Function - Device-independent Block Size

- There are different kind of disk drives. Each may have a different physical sector size.
- A file system uses a block size while mapping files into logical disk blocks.
- This layer can hide the physical sector size of different disks and can provide a fixed and uniform disk block size for higher layers, like the file system.
 - Several sectors can be treated as a single logical block.

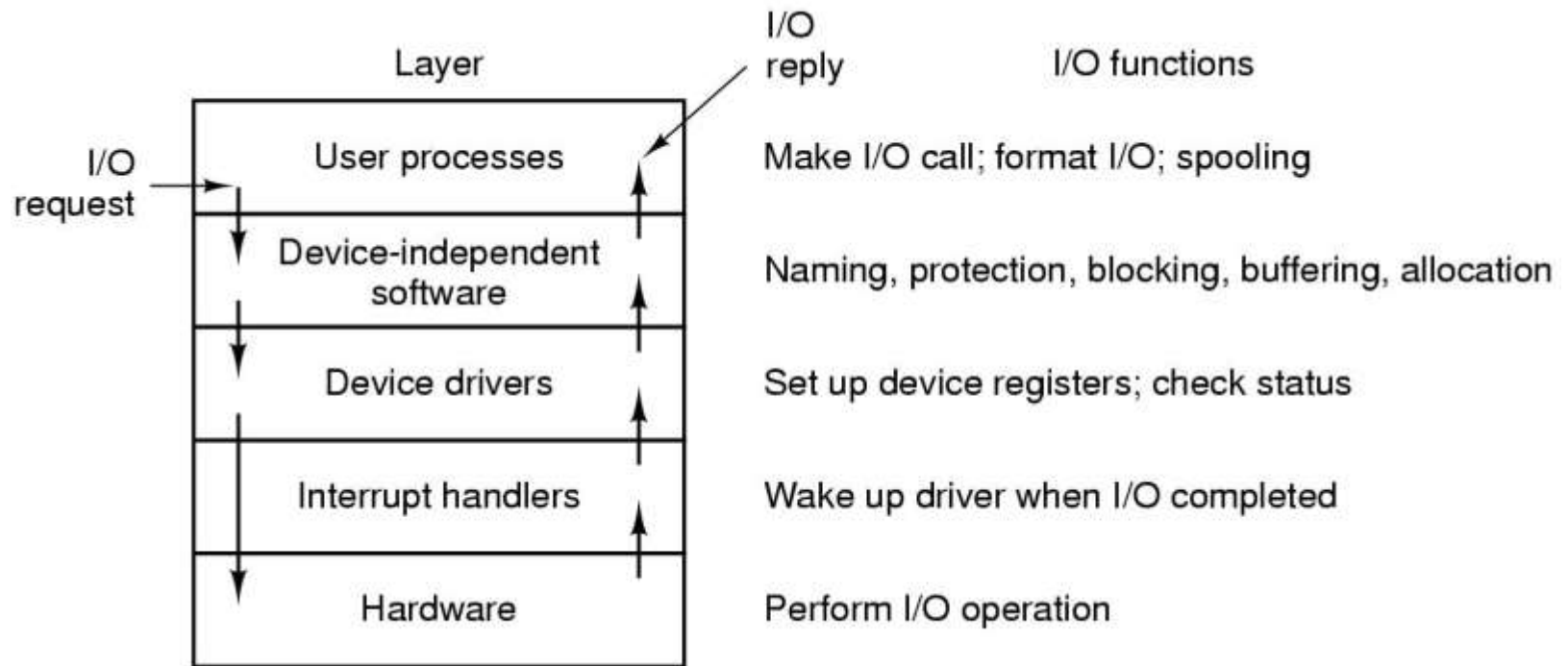
User-Space I/O Software (1)

- These are the libraries which provide richer and simplified interface to access the functionality of the kernel or ultimately interactive with the device drivers.
- Most of the user-level I/O software consists of library procedures with some exception like spooling system which is a way of dealing with dedicated I/O devices in a multiprogramming system.
- I/O Libraries (e.g., stdio) are in user-space to provide an interface to the OS resident device-independent I/O SW.

User-Space I/O Software (2)

- The I/O libraries that provides the implementation of I/O functions which in turn call the respective I/O system calls.
 - These libraries also implemented formatted I/O functions such as *printf()* and *scanf()*.
 - *putchar()*, *getchar()*, *printf()* and *scanf()* are example of user level I/O library *stdio* available in C programming.
 - Some programs that does not directly write to the I/O device, but writes to a spooler.

Summary of I/O Software



Layers of the I/O system and the main functions of each layer



Homework

■ P430 14